

Mobile Learning Application Development



Dart Programming Language Lecture 1

Aditya Rajbongshi
Assistant Professor

Dept. of Educational Technology and Engineering
University of Frontier Technology, Bangladesh



Content

- **Introduction of Dart Programming Language**
- **Dart Environment**
- **Dart SDK Setup**
- **Dart Syntax**
- **Datatype**
- **Variables**
- **Operators**



Dart

❑ Core Of **Flutter Framework**

❑ Programming Language like Java ,C# ,C++

❑ Dart is a **client-optimized programming language** for apps on multiple platforms.

❑ It is developed by **Google** and is used to build mobile, desktop, backend and web applications.

❑ First appeared: October 10, 2011; 11 years ago

❑ Designed by Lars Bak and Kasper Lund

❑ Supports two types of compilation techniques.

❖ **AOT (Ahead of Time)** - It converts the Dart code in the optimized JavaScript code with the help of the dar2js compiler and runs on all modern web-browser. It compiles the code at build time.

❖ **JOT (Just-In-Time)** - It converts the byte code in the machine code (native code), but only code that is necessary



Why Dart ?

❑ The characteristics of Dart in the following point.

- Dart is a **platform-independent language** and supports all operating systems such as Windows, Mac, Linux, etc.
- It is an **open-source language**, which means it available free for everyone. It comes with a BSD license and recognized by the ECMA standard.
- It is an **object-oriented programming language** and supports all features of oops such as inheritance, interfaces, and optional type features.
- Dart is very useful in building real-time applications because of its **stability**.



Dart Installation

❑ Install the Dart SDK on Windows

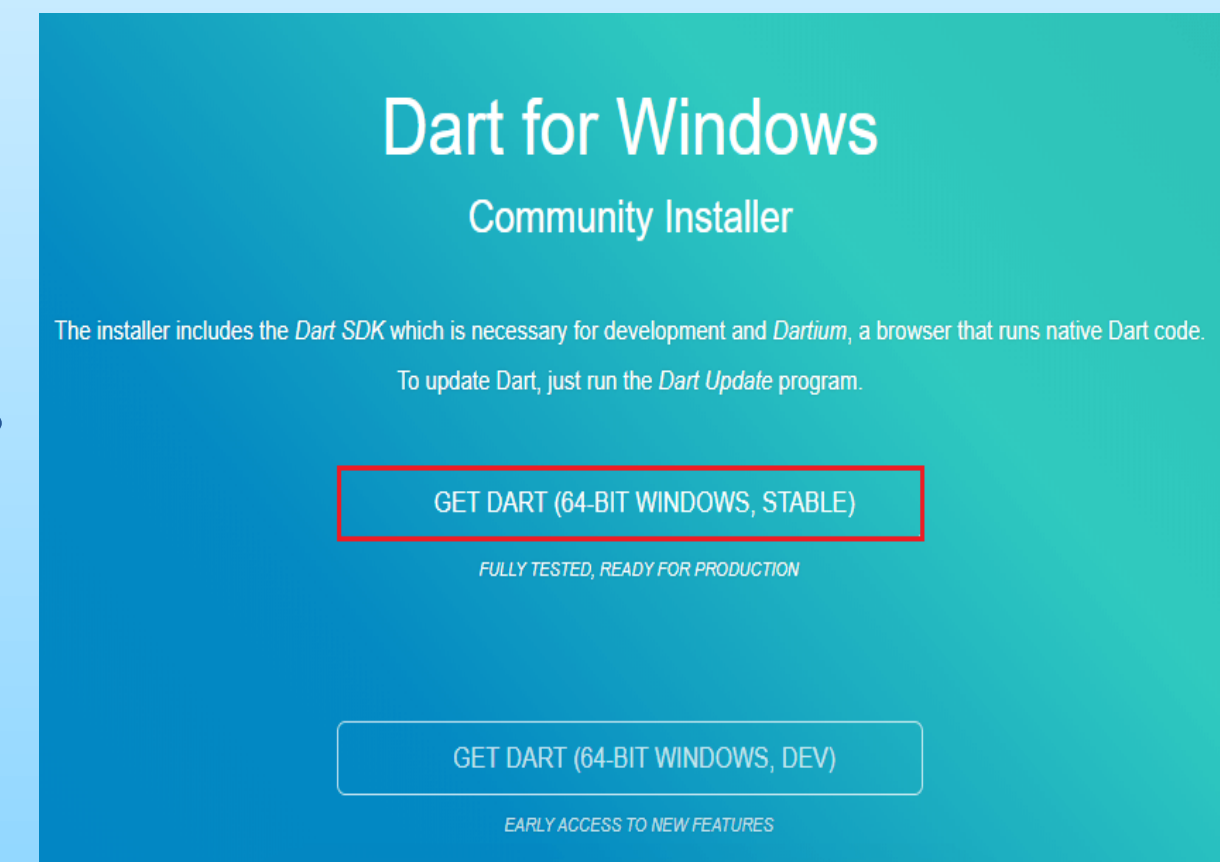
❑ Go to the browser and type the following link to download the SDK.

❑ <http://www.gekorm.com/dart-windows/>

❑ It will open the **given page**. Click on the following link.

❑ Run the **Dart installer**(It is the .exe file that we downloaded in the previous step) and click on the Next button.

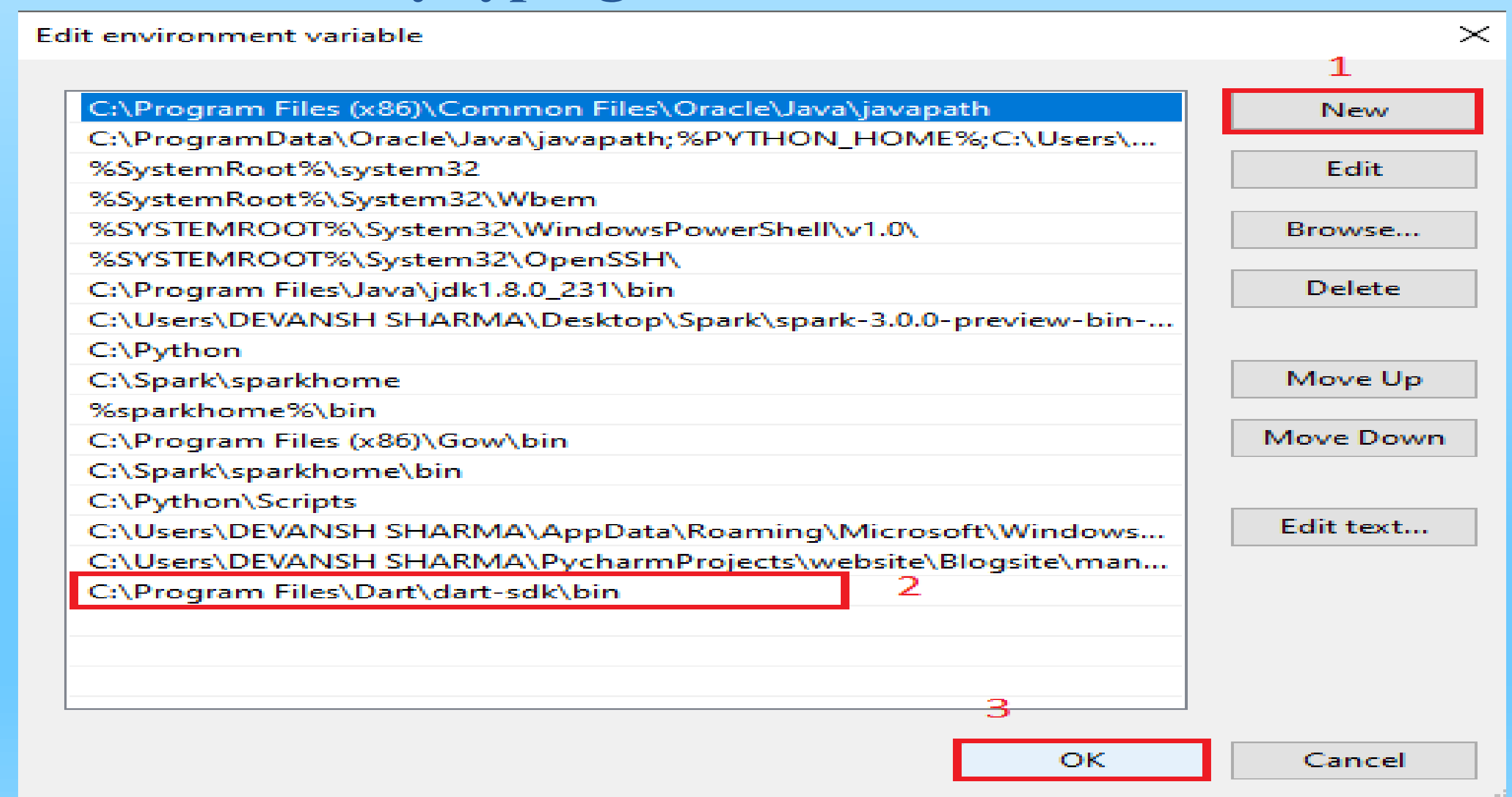
❑ It provides the option to select the **Dart installation path**. After the path is selected, click on the Next button.





Dart Installation

- ❑ After the download is **completed**, set the PATH environment variable to "C:\Program Files\Dart\dart-sdk\bin" in advance system properties.
- ❑ Now open the terminal and verify the Dart installation by typing **dart**.





Dart Environment

❑ **Online Environment: <https://dartpad.dartlang.org>**

❑ **Local Environments:**

❑ **<https://gekorm.com/dart-windows>**

❑ **<https://www.jetbrains.com/idea/download>**



Dart Syntax

Syntax is called set of rules for writing programs.

- Variables and Operators
- Classes
- Functions
- Expression and Programming Constructs
- Decision Making and Looping Constructs
- Comments
- Libraries and Packages
- Data structure represented as Collections / Generics



Dart Syntax

- **Whitespace and Line Breaks**
 - Dart ignores spaces, tabs, and newlines that appear in the program.
- **Dart is Case-sensitive**
 - Dart differentiates between uppercase and lowercase characters.
- **Statements end with semicolon**
 - Each line of instruction is called a statement and a statement must end with a semicolon
- **Comments in Dart**
 - Single-line comments (`//`)
 - Multi-line comments (`/**/`)



Dart Data Types

- **Numbers**
 - Integer: Integer values represent non-fractional values
 - Double: Fractional numeric values
- **Strings**
 - A Dart String is a sequence of UTF-16 code units
- **Boolean**
 - Boolean data types represents true and false
 - Keyword: bool
- **List**
 - List is an ordered group of objects.
 - List is synonymous to the concept of array
- **Map**
 - Map data type represents a set of value as key-value pair
- **Runes**
 - It is used to represent the special syntax.



Dart Variables

- ❑ A variable is a named space in the memory that stores values. In other word variable is a container for the values in a program
- ❑ **Rule to Create Variable**
- ❑ The variable **cannot contain special characters** such as whitespace, mathematical symbol, runes, Unicode character, and keywords.
- ❑ The first character of the variable should be an **alphabet**([A to Z],[a to z]). **Digits are not** allowed as the first character.
- ❑ Variables are **case sensitive**. For example, - variable age and AGE are treated differently.
- ❑ The **special character** such as #, @, ^, &, * are not allowed expect the underscore(_) and the dollar sign(\$).
- ❑ The **variable name** should be retable to the program and readable.



Dart Variables

- **Syntax:** `var <variable_name> = <value>;`
- **Keyword:** `dynamic`

A screenshot of the DartPad web editor interface. The editor has a dark theme. At the top, there's a header with the DartPad logo, navigation links like 'New Pad', 'Reset', 'Format', and 'Install SDK', a user profile 'indescent-rauna-3037', and a 'Samples' dropdown. The main area is split into two panes. The left pane contains a Dart code snippet:

```
1 void main() {  
2   dynamic x= 30;  
3   print(x);  
4   x="hello world";  
5   print(x);  
6 }  
7
```

A blue 'Run' button is positioned to the right of the code. The right pane is divided into two sections: 'Console' and 'Documentation'. The 'Console' section shows the output of the program: '30' and 'hello world'. The 'Documentation' section shows the definition for 'dynamic x' as a 'local variable'. At the bottom of the editor, there's a footer with links for 'Privacy notice', 'Send feedback', 'stable channel', 'no issues', and 'Based on Flutter 3.7.0 Dart SDK 2.19.0'.



Dart Variables

- **final vs const**
 - **final:** The final keyword is used to hardcode the values of the variable and it cannot be altered in future, neither any kind of operations performed on these variables can alter its value (state).

The screenshot shows the DartPad web IDE interface. The editor contains the following Dart code:

```
1 void main() {  
2   final x= 30;  
3   print(x);  
4   x=32;  
5   print(x);  
6 }  
7
```

A blue "Run" button is visible next to the code. The console on the right displays the following output:

```
Error compiling to JavaScript:  
Info: Compiling with sound null safety.  
lib/main.dart:4:3:  
Error: Can't assign to the final variable 'x'.  
  x=32;  
  ^  
Error: Compilation failed.
```

Below the console, a documentation panel shows an error message for line 4:

```
error line 4 • The final variable 'x' can only be set  
once. (view docs)  
Try making 'x' non-final.
```

The bottom status bar indicates "Based on Flutter 3.7.0 Dart SDK 2.19.0" and shows "1 issue".



Dart Variables

- **final vs const**
 - **const:** The Const keyword in Dart behaves exactly like the final keyword. The only difference between final and const is that the const makes the variable constant from compile-time only. Using const on an object, makes the object's entire deep state strictly fixed at compile-time and that the object with this state will be considered frozen and completely immutable.

The screenshot shows the DartPad web IDE interface. The code editor on the left contains the following Dart code:

```
1 void main() {  
2   const x = 30;  
3   print(x);  
4   x = 32;  
5   print(x);  
6 }  
7
```

A blue "Run" button is visible next to the code. The right-hand side of the interface is split into two panels. The top panel, labeled "Console", shows the following error message:

```
Error compiling to JavaScript:  
Info: Compiling with sound null safety.  
lib/main.dart:4:3:  
Error: Can't assign to the const variable 'x'.  
  x=32;  
  ^  
Error: Compilation failed.
```

The bottom panel, labeled "Documentation", shows an error tooltip for line 4:

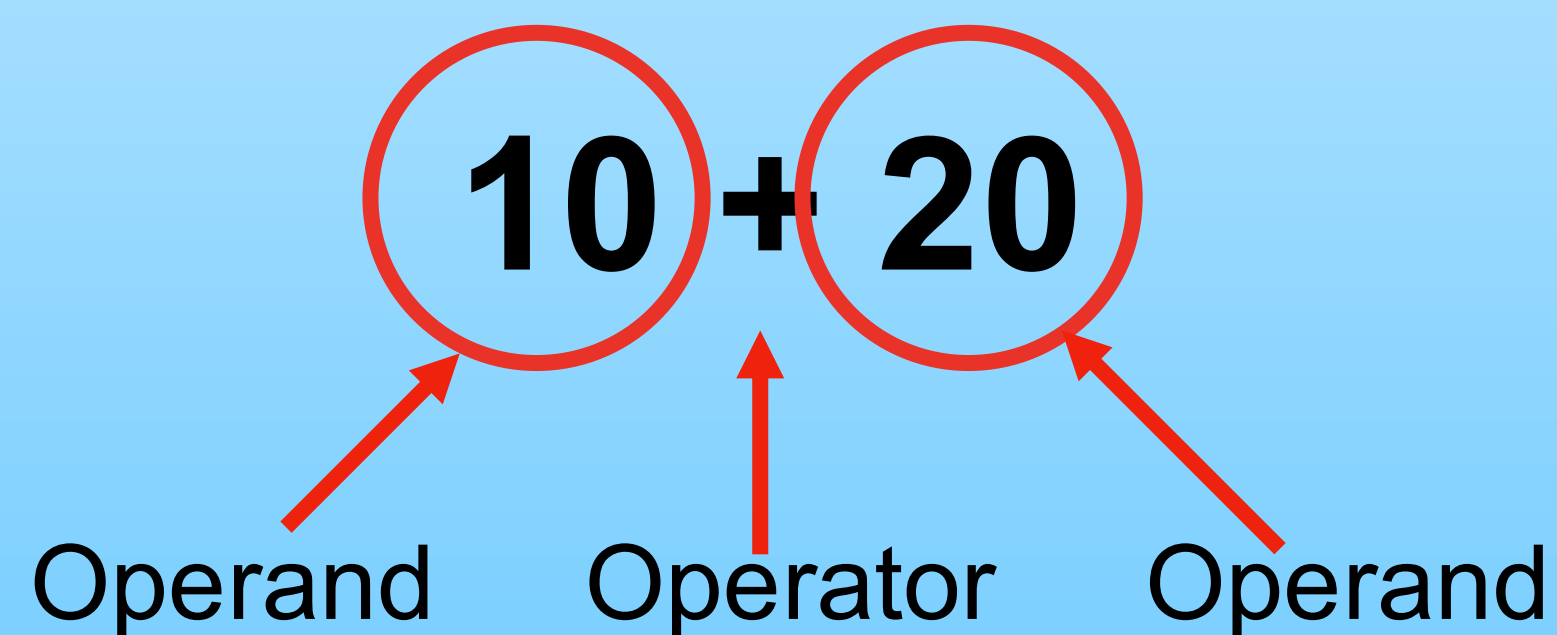
```
error line 4 • Constant variables can't be assigned  
a value. (view docs)  
Try removing the assignment, or remove the  
modifier 'const' from the variable.
```

The bottom status bar of the IDE indicates "Based on Flutter 3.7.0 Dart SDK 2.19.0" and shows "1 issue".



Dart Operators

- **Operands** represents data
- **Operators** defines how the operands will be processed to produce a value



- **Arithmetic Operators:** +, -, *, /, %, ++, - -
- **Relational Operators:** <, >, <=, >=, ==, !=
- **Assignment Operators:** =, ??=, -=, +=, *=, /=, %=
- **Logical Operators:** &&, ||, !



Dart Data Type: String

- The String data type represents **a sequence of characters**
- A Dart string is a sequence of **UTF 16 code units**.
- String values in Dart can be represented using
 - Single quotes
 - Double quotes
 - Triple quotes
- **Single line strings are represented using single or double quotes.**
- **Triple quotes are used to represent multi-line strings.**



Dart Data Type: String

- **String Concatenation**

- The `+` or `+=` operator is used to merge the two string.

```
1 ▼ void main() {  
2   String a= "Hello ";  
3   String b= "World!";  
4   String c= a+b;  
5   print(c);  
6 }  
7
```

▶ Run

Console

Hello World!



Dart Data Type: String

- **String Interpolation**

- The string interpolation is a technique to manipulate the string and create the new string by adding another value.
- It can be used to evaluate the string including placeholders, variables, and interpolated expression.
- The `{expression}` is used for string interpolation.
- The expressions are replaced with their corresponding values.



Dart Data Type: String Methods

- **toLowerCase():** It converts all characters of the given string in lowercase.
- **toUpperCase():** It converts all characters of the given string in uppercase.
- **trim():** It eliminates all whitespace from the given string.
- **compareTo():** It compares one string from another.
- **replaceAll():** It replaces all substring that matches the specified pattern with a given string.
- **split():** It splits the string at matches of the specified delimiter and returns the list of the substring.
- **substring():** It returns the substring from start index, inclusive to end index.
- **toString():** It returns the string representation of the given object.
- **codeUnitAt():** It returns the 16-bits code unit at the given index.